SAPPHIRE PROGRAMMING LANGUAGE

Complete Coding Manual, Syntax Reference & System Library Guide

1. Introduction to Sapphire Language

Sapphire is a modern high-level programming language engineered specifically for **PC Automation**, **Autonomous AI Workflows**, and **Colorless Parallel Concurrency**. Sapphire combines clean, readable syntax resembling Rust and JavaScript with zero-boilerplate standard library primitives for OS manipulation, web access, data serialization, and AI prompts.

> **Core Philosophy: Zero Boilerplate Autonomy**
>
> Unlike Python which requires heavy setup (pip, external drivers, complex async loops), Sapphire embeds system automation, scheduler tasks, and AI primitives natively into the language runtime.

2. Language Syntax & Fundamentals

2.1 Variable Bindings & Types

Variables in Sapphire are declared using the let keyword. Sapphire is dynamically typed with optional type hint syntax.

```
let agent_name = "Sapphire-Alpha";
let version = 1.0;
let is_active = true;
let system_metrics = [95.4, 88.1, 91.0];
let config = {"retry": 3, "timeout": 30};

print("Agent {agent_name} running version {version}");
```

2.2 String Interpolation

Sapphire supports inline string interpolation directly within double-quoted strings using {expression} format. Complex expressions and array properties can be evaluated seamlessly.

2.3 Control Flow & Functions

```
fn calculate_health_index(cpu, ram) {
    if (cpu > 90 or ram > 90) {
        return "CRITICAL";
    } else if (cpu > 70) {
        return "WARNING";
    } else {
        return "HEALTHY";
    }
}

let status = calculate_health_index(85, 42);
print("System Status: {status}");
```

3. Native Standard Libraries

3.1 os Module

Provides system telemetry, clipboard read/write, native OS notifications, and command execution.

```
let info = os.system_info();
os.notify('Alert', 'CPU spike detected: {info.cpu_usage_percent}%');
os.clip_write('Telemetry exported');
```

3.2 fs Module

Provides high-level file system reading, writing, directory listing, and file checks.

```
let files = fs.list_dir('./logs');
fs.write('./status.txt', 'All systems operational');
```

3.3 http Module

HTTP request client for GET, POST, JSON API integration.

```
let resp = http.get('https://api.github.com/zen');
if resp.ok { print('GitHub Zen: {resp.text}'); }
```

3.4 ai Module

First-class AI primitive for zero-boilerplate LLM prompts and intelligence evaluation.

```
let summary = ai.prompt('Summarize system logs: {raw_logs}');
print('AI Summary: {summary}');
```

3.5 scheduler Module

Cron and background interval scheduling for persistent autonomous loops.

```
scheduler.interval(5.0, fn() {
    print('Executing periodic autonomous audit...');
});
```

4. Parallel Concurrency (`parallel` block)

Sapphire introduces **Colorless Concurrency**. Instead of marking functions as async and placing await keywords on every call, Sapphire uses a simple parallel block to run independent statements concurrently.

```
print("■ Starting parallel PC diagnostic tasks...");

parallel {
    print("Task 1: Fetching network diagnostics...");
    print("Task 2: Scanning disk storage integrity...");
    print("Task 3: Auditing RAM memory footprint...");
}

print("■ All parallel tasks completed!");
```

5. Complete Production Code Example

```
// Autonomous PC Monitoring Agent in Sapphire
fn run_pc_autobot() {
    print("■ Launching Sapphire PC Autobot...");

    // 1. Audit System Info
    let stats = os.system_info();
    print("System Diagnostics: RAM {stats.ram_percent}%, CPU {stats.cpu_usage_percent}%");

    // 2. Parallel Network & Disk Audit
    parallel {
        let http_check = http.get("https://httpbin.org/get");
        print("Network check status: {http_check.status_code}");

        let files = fs.list_dir(".");
        print("Workspace contains {files.length} items.");
    }

    // 3. AI Evaluation & Action
    let ai_assessment = ai.prompt("System RAM load is {stats.ram_percent}%. Give short 1-li
ne advice.");
    print("AI Evaluation: {ai_assessment}");

    // 4. Toast Notification
    os.notify("Sapphire Agent", "PC Health Check Completed!");
}

run_pc_autobot();
```